Recursive and Iterative Inorder Comparison

1. Problem Statement

To write a C++ program to implement recursive and iterative inorder traversal of a binary tree and verify that both methods produce identical output. The program must automatically compare the traversal results and print PASS or FAIL, measure the maximum stack size used by the iterative method, and test empty, single-node, balanced, left-skewed, and right-skewed binary trees.

2. Objective

1. To implement recursive inorder traversal using function calls.
2. To implement iterative inorder traversal using a stack.
3. To compare both traversal outputs automatically and display PASS or FAIL.
4. To measure the maximum stack size used during iterative traversal.
5. To test the program on empty, single-node, balanced, left-skewed, and right-skewed

3. Algorithm

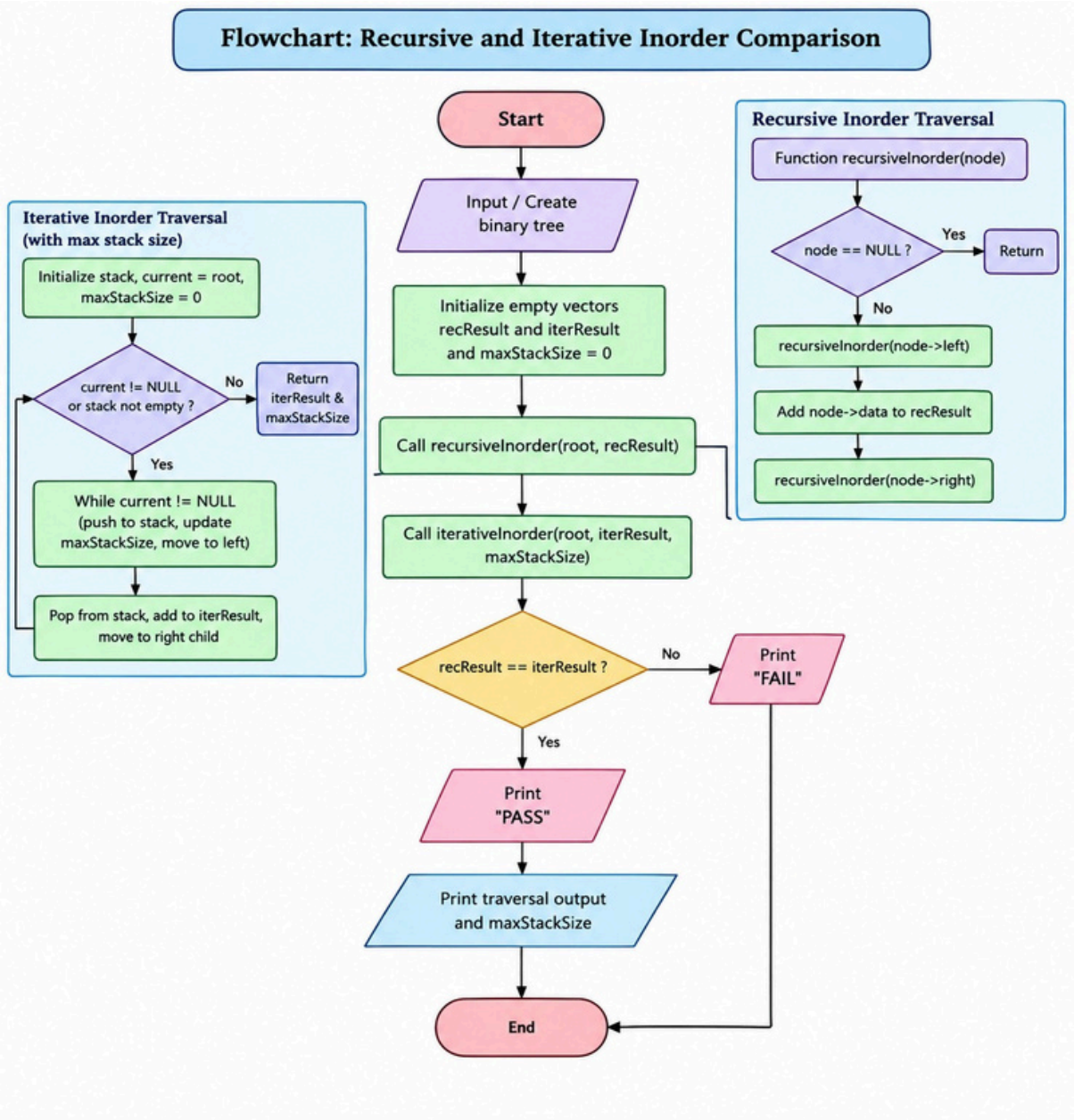
1. Start.
2. Create a binary tree for the selected test case.
3. Initialize two empty vectors, recResult and iterResult, and set maxStackSize = 0.
4. **Recursive inorder:** If the current node is NULL, return. Otherwise, traverse the left subtree, store the current node's data, and traverse the right subtree.
5. **Iterative inorder:** Initialize an empty stack and set current = root. While current is not NULL or the stack is not empty, push nodes while moving left and update maxStackSize. Pop a node, store its data, and move to its right child.
6. Compare recResult and iterResult. If they are equal, print PASS; otherwise, print FAIL. 7.

Display both traversal outputs and the maximum iterative stack size.

8. Repeat for empty, single-node, balanced, left-skewed, and right-skewed trees.
9. Stop.

4. Flowchart

Flowchart showing the traversal steps, automatic result comparison, and maximum stack-size measurement.



5. Conclusion

The program verifies that recursive and iterative inorder traversal produce identical results for different binary tree structures. It also reports PASS or FAIL automatically and measures the maximum stack size used by the iterative method. Both approaches take O(n) time, while their auxiliary space is O(h), where n is the number of nodes and h is the height of the tree.